Algorithms and Data Structures using Java

Lab Exam Practice Question Bank

15 Direct Topic-Based Programming Questions

No scenario-based questions. Questions are framed directly from the DSA topics in the provided syllabus.

Coverage

Area	Topics included
Linear Data Structures	Arrays, Stack, Queue, Circular Queue, Linked Lists
Recursion & Trees	Recursion, BST, AVL Tree, traversals and deletion
Searching & Sorting	Linear/Binary Search, elementary sorts, Quick/Merge/Heap Sort
Hashing	Hash functions and collision resolution
Graphs	Representations, BFS, DFS, Dijkstra, Floyd-Warshall, Prim, Kruskal
Algorithm Design	Greedy, DP, Backtracking, complexity analysis

Instruction: Write Java programs using your own classes/methods. Unless explicitly permitted, implement the required data structure or algorithm logic instead of using Java library classes that directly solve it.

Question 1 - Stack Using Array

Topic focus: Arrays, Stack ADT, time complexity

Task

- Implement a Stack class using an integer array and an integer top index.
- Provide push(int), pop(), peek(), isEmpty(), isFull() and display() operations.
- Handle stack overflow and stack underflow conditions with suitable messages.
- Demonstrate the implementation through a menu-driven main program.
- State the time complexity of push(), pop() and peek().

Question 2 - Queue and Circular Queue Using Array

Topic focus: Queue, Circular Queue, wrap-around indexing

Task

- Part A: Implement a linear queue using an array with enqueue(), dequeue(), front() and display().
- Part B: Implement a circular queue using an array with enqueue(), dequeue(), front(), isEmpty() and isFull().
- Use front and rear indices correctly so that vacant positions can be reused in the circular queue.
- Demonstrate overflow, underflow and normal insertion/deletion cases.

Question 3 - Singly Linked List Operations

Topic focus: Singly linked list, node manipulation

Task

- Create a singly linked list of integers using a Node class.
- Implement insertion at beginning, insertion at end and insertion at a given position.
- Implement deletion from beginning, deletion from end and deletion by value.
- Implement search(), display() and reverse() operations.
- Display the list after every modification.

Question 4 - Doubly and Circular Linked Lists

Topic focus: Doubly linked list, circular linked list

Task

- Part A: Implement a doubly linked list with insert-at-end, delete-by-value, forward display and backward display.
- Part B: Implement a circular singly linked list with insertion at beginning, insertion at end, deletion and traversal.
- Clearly maintain prev/next links in the doubly linked list and the last-to-first link in the circular list.

Question 5 - Stack and Queue Using Linked List

Topic focus: Linked-list based stack and queue

Task

- Implement a stack using a linked list. Provide push(), pop(), peek() and display().
- Implement a queue using a linked list. Provide enqueue(), dequeue(), front() and display().
- Do not use java.util.Stack, Queue, LinkedList or ArrayDeque for the actual data-structure implementation.
- State the time complexity of each insertion and deletion operation.

Question 6 - Recursive Algorithms

Topic focus: Recursion, base condition, recursive complexity

Task

- Write recursive methods for factorial of n, nth Fibonacci number and GCD of two integers.
- Write a recursive binary search method for a sorted integer array.

- For each method, identify the base condition and the recursive call.
- Write the time complexity of factorial, naive Fibonacci and recursive binary search.

Question 7 - Binary Search Tree

Topic focus: BST creation, search, deletion, DFS/BFS traversals

Task

- Implement a Binary Search Tree for integer keys.
- Provide insert(), search() and delete() operations.
- Implement inorder, preorder and postorder traversals recursively.
- Implement breadth-first traversal using a queue.
- Test deletion of a leaf node, a node with one child and a node with two children.

Question 8 - AVL Tree Insertion

Topic focus: AVL tree, balance factor, rotations

Task

- Implement insertion in an AVL tree for integer keys.
- Calculate the height and balance factor of every affected node after insertion.
- Implement LL, RR, LR and RL rotations.
- Print preorder traversal after all insertions so the balanced structure can be verified.

Question 9 - Linear Search and Binary Search

Topic focus: Searching, correctness and complexity

Task

- Implement iterative linear search for an unsorted integer array.
- Implement iterative and recursive binary search for a sorted integer array.
- For a given key, print its index if found; otherwise print an appropriate not-found message.
- Count and display the number of comparisons made by linear search and binary search for the same searchable key.
- State best-case and worst-case time complexity for both search methods.

Question 10 - Sorting Algorithms

Topic focus: Selection, insertion, bubble, quick, merge and heap sort

Task

- Create separate Java methods for selection sort, insertion sort and bubble sort.
- Create separate methods for quick sort, merge sort and heap sort.
- Accept the same integer array and allow the user to choose a sorting algorithm from a menu.
- Print the sorted array and the theoretical time complexity of the selected algorithm.
- For bubble/insertion/selection sort, also count comparisons or swaps to observe practical behavior on the given input.

Note: The purpose is implementation and comparison; built-in sorting methods should not be used.

Question 11 - Hash Table with Collision Resolution

Topic focus: Hash tables, linear probing, quadratic probing, double hashing

Task

- Implement a hash table for integer keys using an array.
- Use key % tableSize as the primary hash function.
- Implement insertion and search using linear probing.
- Extend the program to support quadratic probing and double hashing as alternative collision-resolution methods.
- Provide delete() using a suitable deleted-marker approach so search remains correct after deletion.

Question 12 - Graph Representation and Traversal

Topic focus: Adjacency matrix, adjacency list, BFS, DFS

Task

- Represent an undirected graph using an adjacency matrix.
- Represent the same graph using an adjacency list.
- Implement Depth First Search starting from a given vertex.
- Implement Breadth First Search starting from a given vertex.
- Ensure every vertex is visited exactly once and support disconnected graphs by traversing remaining unvisited vertices.

Question 13 - Shortest Path Algorithms

Topic focus: Dijkstra and Floyd-Warshall

Task

- Part A: Implement Dijkstra's algorithm for a weighted graph with non-negative edge weights and print the shortest distance from a source vertex to every other vertex.
- Part B: Implement the Floyd-Warshall algorithm and print the all-pairs shortest-path distance matrix.
- Use a sufficiently large value to represent infinity where no direct edge exists.

- Compare the type of shortest-path problem solved by Dijkstra and Floyd-Warshall.

Question 14 - Minimum Spanning Tree

Topic focus: Prim and Kruskal algorithms, greedy technique

Task

- Implement Prim's algorithm for a connected weighted undirected graph.
- Implement Kruskal's algorithm for the same graph.
- For Kruskal's algorithm, use a disjoint-set/union-find structure or an equivalent cycle-detection method.
- Print the selected MST edges and the total MST weight for both algorithms.
- State the main data structure used by each algorithm and its time complexity.

Question 15 - Algorithm Design Techniques

Topic focus: Greedy, dynamic programming, backtracking, asymptotic analysis

Task

- Part A - Greedy: Implement activity selection or fractional knapsack and print the selected activities/items.
- Part B - Dynamic Programming: Implement 0/1 knapsack or Longest Common Subsequence using a DP table.
- Part C - Backtracking: Implement N-Queens or subset-sum and print one valid solution if it exists.
- For each implemented part, write the recurrence/choice rule used and state the time and space complexity.
- Briefly identify why the chosen problem fits greedy, dynamic programming or backtracking respectively.

Note: For practice, solve at least one problem from each of the three design techniques.

End of Practice Question Bank